

## SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

### CO3 ASSESSMENT TOOL 2 – Git & Docker Technical Assignment

|                |                                                                                                                    |               |                                                        |
|----------------|--------------------------------------------------------------------------------------------------------------------|---------------|--------------------------------------------------------|
| Course Code:   | CSA1007                                                                                                            | Course Title: | Software Engineering                                   |
| CO Assessed:   | CO3                                                                                                                | Assessment:   | Assessment Tool 2 – Code Review Repository Evaluation. |
| Weightage:     | 20%                                                                                                                | Total Marks:  | 25                                                     |
| CO3 Statement: | Build full-stack applications with an emphasis on containerization, version control, and collaborative development |               |                                                        |
| Student Name:  | G. Akshya                                                                                                          | Reg. No.:     | 192425132                                              |
| Date:          | 15/08/2026                                                                                                         | Duration:     | 60 minutes                                             |

#### Code of Conduct

*I G. Akshya (192425132) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: G. Akshya

#### Annexure A – Git & Docker Technical Assignment

## Co 3- Assessment Tool-02

**Name:** -G. Akshya

**Reg.No:** -192425132

**Course Code:** -CSA1007

**Slot:** -D

### 1. Abstract

The Smart Vertical Farming Automation Platform is a Flask-based web application developed to demonstrate centralized monitoring and automation information for a vertical farming environment. The dashboard presents environmental values such as temperature, humidity, soil moisture, and light intensity, together with automation states, crop health, and alerts.

Git and GitHub were used for version control, including repository initialization, meaningful commits, branch-based development, feature integration, Git history, and remote synchronization. Docker was used to containerize the application, while Docker Compose was used to build and run the service. The completed application was successfully accessed through localhost:5000 and the final develop branch was pushed to GitHub.

### 2. Problem Statement

Smart vertical farms require continuous observation of environmental conditions and efficient management of farm resources. A digital monitoring interface can provide a single place to view important conditions and automation states. The project demonstrates a lightweight web platform for this purpose while also demonstrating a professional software-development workflow using Git and Docker.

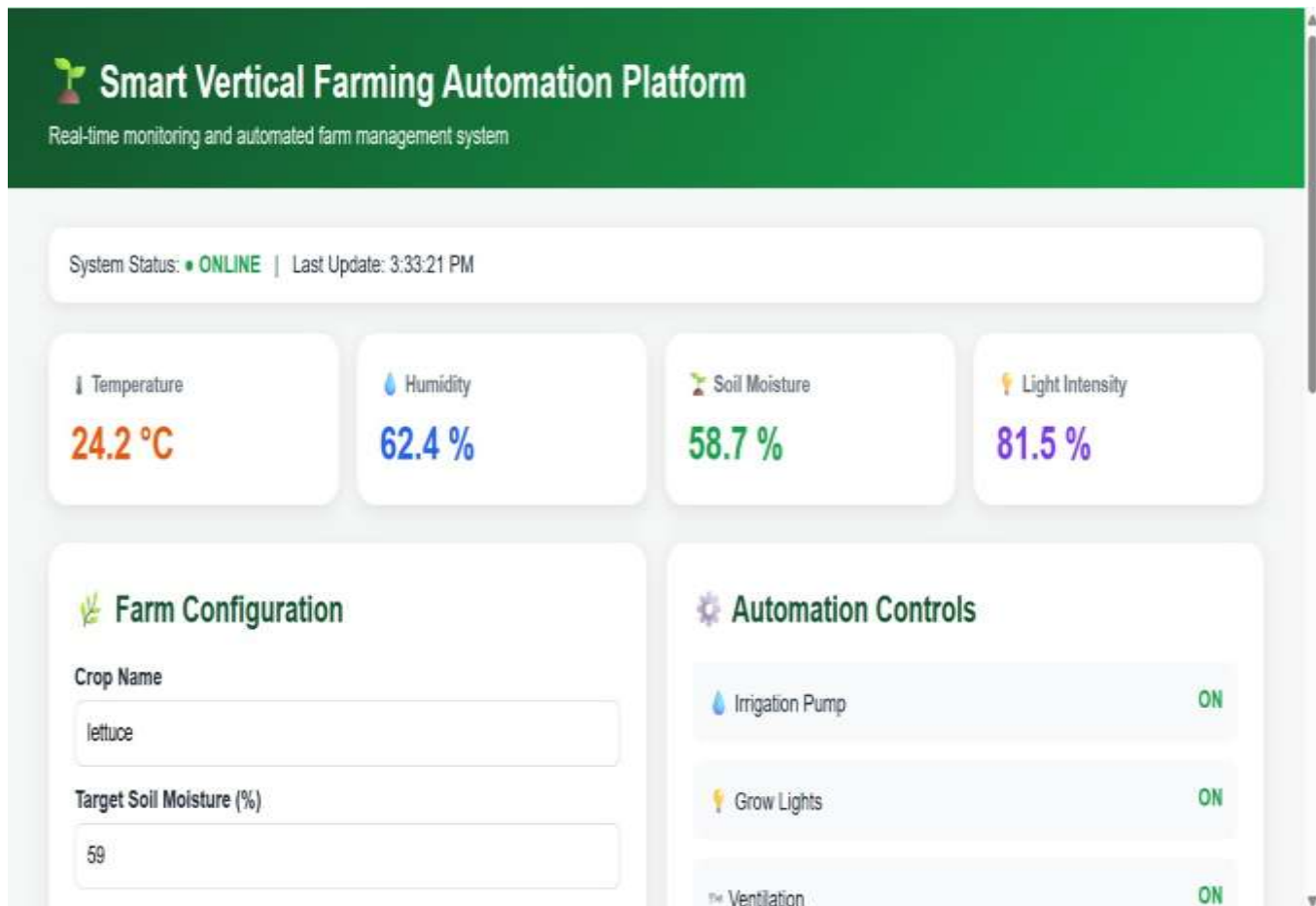
### 3. Objectives

- Develop a clear web dashboard for a smart vertical farming scenario.
- Implement the web application using Python Flask and HTML/CSS.
- Demonstrate branching and merging using a dashboard feature branch.
- Containerize the application using Docker.
- Use Docker Compose for repeatable local deployment.
- Verify the application through a browser and container-status checks.
- Maintain the final project in a GitHub remote repository.

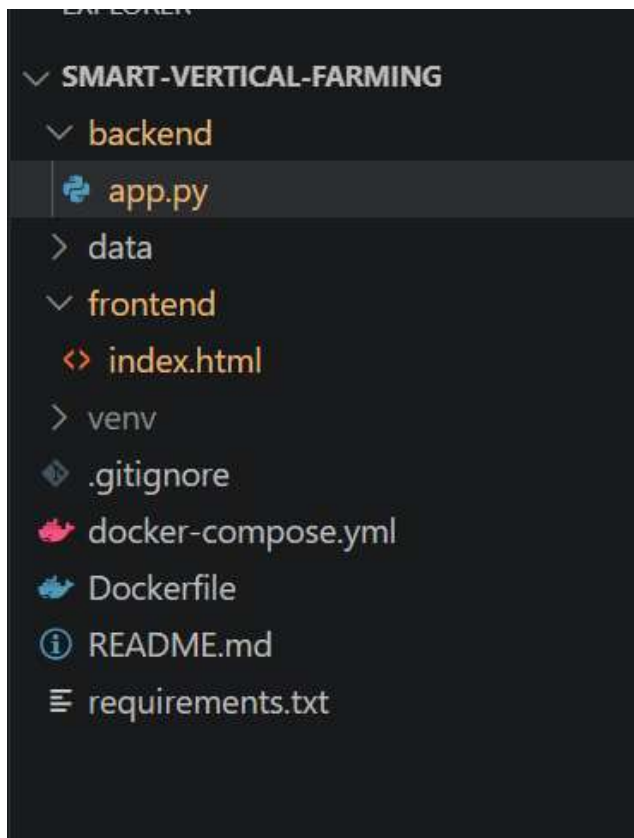
### 4. Technologies Used

| Technology / Tool | Purpose                   | Version / Details                |
|-------------------|---------------------------|----------------------------------|
| Python            | Backend programming       | 3.14.2                           |
| Flask             | Web application framework | Installed in virtual environment |
| HTML / CSS        | Frontend dashboard        | Responsive custom interface      |

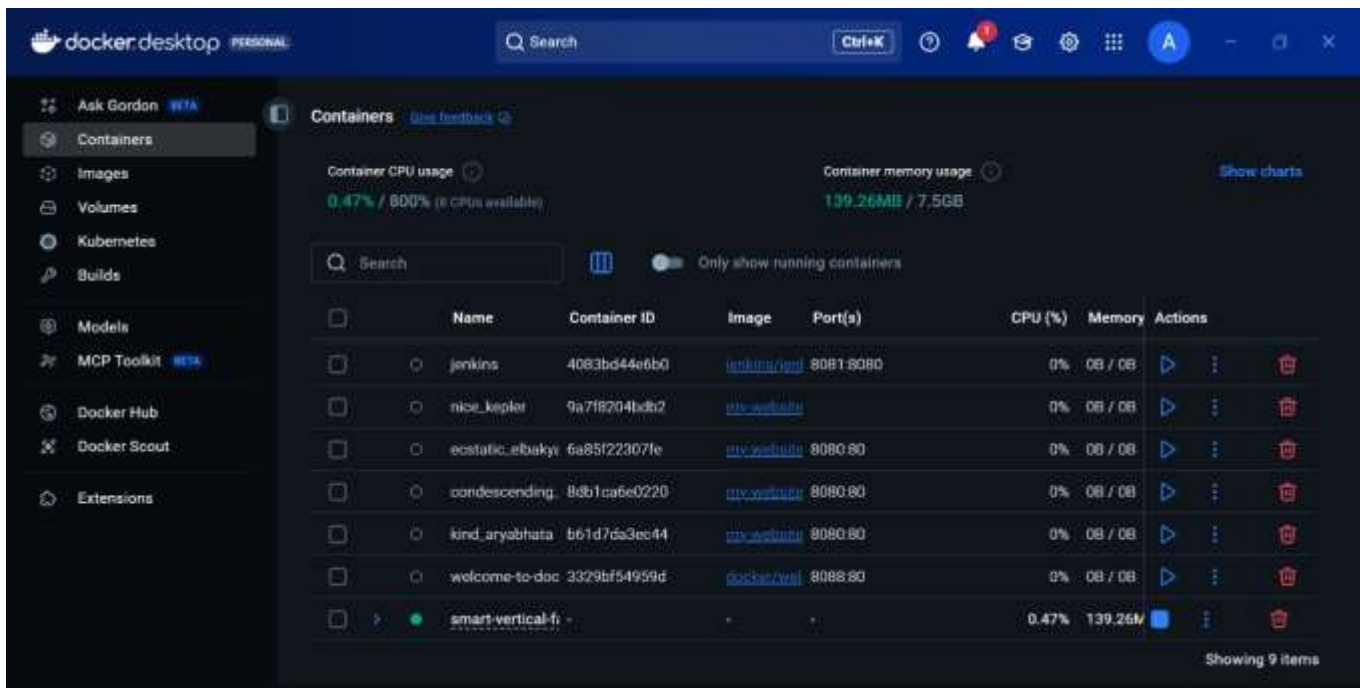
|                |                         |                              |
|----------------|-------------------------|------------------------------|
| Git            | Version control         | 2.55.0. windows.3            |
| GitHub         | Remote repository       | develop branch pushed        |
| Docker         | Containerization        | Docker Desktop               |
| Docker Compose | Build and run service   | smart-farming service        |
| VS Code        | Development environment | Project editing and terminal |



**Fig 1: -Dashboard**



**Fig 2: -Git Hub Repository**



**Fig 3: -Web Site Running in Docker**

The backend contains the Flask application. The frontend contains the dashboard. The data folder is reserved for project data. The virtual environment contains local Python dependencies. Docker file and docker-compose.yml provide containerized deployment configuration.

Figure 1. System Architecture — replace or supplement this text diagram with the final architecture graphic in the submitted report.

## 5. Application Workflow

- ❖ The project is developed and maintained in VS Code.
- ❖ Git tracks the application source code and configuration files.
- ❖ The Flask backend creates the web application and serves the dashboard.
- ❖ The Docker file defines the runtime environment and application startup command.
- ❖ Docker Compose builds the application image and starts the smart-farming service.
- ❖ Port 5000 of the container is mapped to port 5000 on the host computer.
- ❖ The user opens `http://localhost:5000/` in a web browser.
- ❖ The dashboard displays environmental monitoring, automation status, crop health, and alerts.

## 6. Frontend Dashboard

The frontend is a custom HTML/CSS dashboard designed for a modern smart-agriculture presentation. It contains a live-status indicator, environmental sensor cards, automation controls, crop-health information, and alert information.

| Dashboard Area           | Displayed Information                                 |
|--------------------------|-------------------------------------------------------|
| Environmental Monitoring | Temperature, humidity, soil moisture, light intensity |
| Automation Control       | Irrigation, grow lighting, ventilation                |
| Crop Health              | Health score and Healthy status                       |
| Alerts                   | Current alert status                                  |
| Farm Visual              | Illustrative vertical-farm levels                     |

Current dashboard values displayed by the interface include temperature 25 °C, humidity 65%, soil moisture 42%, light intensity 78%, irrigation ON, lighting ON, ventilation ON, and crop health Healthy.

## 7. Docker Implementation

Docker is used to package the Flask application and its required Python environment into a container. This makes the application easier to run consistently without depending on the host machine's Python configuration.

## 8. Where Docker Was Used

- Docker file: defines how the application image is built.
- Docker Compose: defines how the application container is run.
- Docker image: packages the application and dependencies.
- Docker container: runs the Flask application.
- Port mapping: connects host port 5000 to container port 5000.
- Browser: accesses the running container through `http://localhost:5000/`.

## 9. Advantages

- ❖ Simple and clear monitoring interface.
- ❖ Centralized presentation of farming parameters.
- ❖ Separation of backend and frontend components.
- ❖ Git-based development and version history.
- ❖ Feature branch workflow supports organized development.
- ❖ Docker provides a consistent application environment.
- ❖ Docker Compose simplifies application startup.
- ❖ Application can be demonstrated locally through a browser.

## 10. Limitations

The displayed environmental values are currently dashboard values rather than readings from connected physical sensors.

The project does not currently include a persistent database.

The automation controls are presented as dashboard states rather than connected actuator commands.

The current deployment is local rather than cloud-hosted.

## 11. Conclusion

The Smart Vertical Farming Automation Platform demonstrates the complete lifecycle of a small software project: application development, version control, feature branching, merging, containerization, deployment, testing, and remote repository management. The project successfully combines a Flask backend with a professional frontend dashboard and uses Docker and Docker Compose to provide a consistent deployment environment. The completed Git workflow and Docker deployment satisfy the main technical requirements of the assignment and provide a strong foundation for future IoT and automation enhancements.